Match Statement

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY – INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise

Imagine you're participating in a Mario Kart race with your friends. Each player finishes the race in a specific position.

Write a Python program that takes as input the finishing position of a player, given as a positive integer, and returns the position in cardinal form (e.g., "1st", "2nd", "3rd", "4th", "5th", etc.).

Expected Output:

Finishing position: 1

1st!

Finishing position: 2

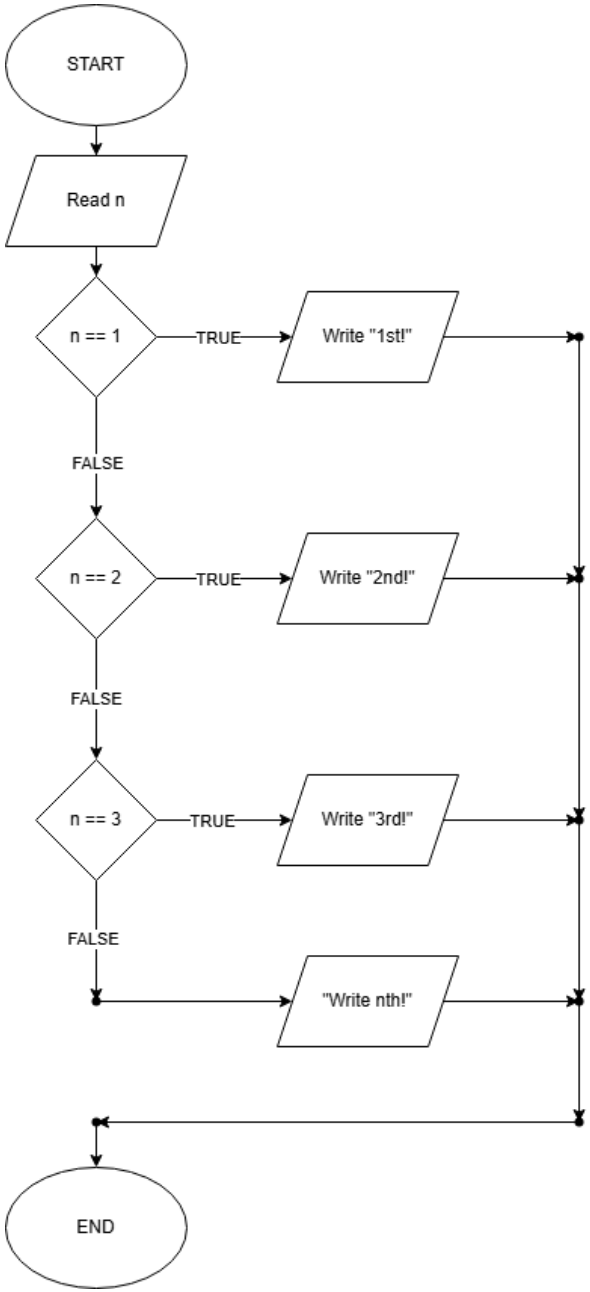
2nd!

Finishing position: 4

4th!

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Solution/1



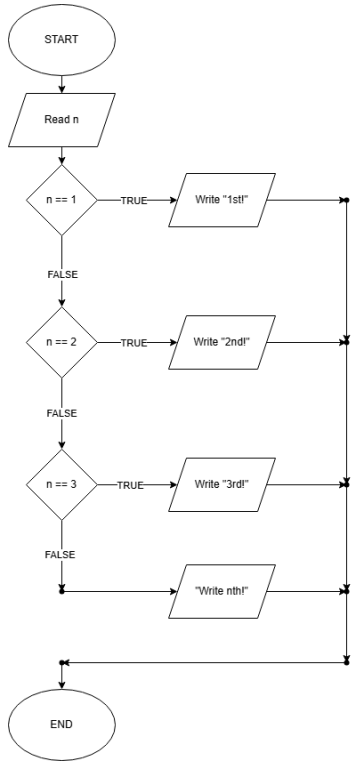
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Solution/2

```
# insert a number
n:int = int(input("Insert Finishing Position: "))

# check inserted number and return the position in cardinal form

# if n=1
if n == 1:
    print(f"{n}st!")
# if n=2
elif n == 2:
    print(f"{n}nd!")
# if n=3
elif n == 3:
    print(f"{n}rd!")
# else
else:
    print(f"{n}th!")
```



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement/1

- An algorithm often requires decisions based on comparing a variable with its possible values.
- Different actions are executed for each value.

- Blocks with many **if-elif-else** statements become **hard to read**.
- The complexity of code understanding increases as the number of alternative paths grows.

- The **match** statement improves code readability and comprehension.
- Provides a cleaner structure for managing multiple decisions.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement/2

- The **match** statement **evaluates** the **variable value**.

- The list of **case** options is checked **one by one** until a **match** with the **variable value** is found.
- When a **matching case** is found, the associated **action** is **executed**.

- If **no case** matches, the **default case** (_) is **executed**.

```
match variable:
    case variable_value1:
            instructions for variable_value1
    case variable_value2:
            instructions for variable_value2
    ...
    case _:
            instruction for default case
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise

We can rewrite this code using match statement or a more readable solution.

Imagine you're participating in a Mario Kart race with your friends. Each player finishes the race in a specific position.

Write a Python program that takes as input the finishing position of a player, given as a positive integer, and returns the position in cardinal form (e.g., "1st", "2nd", "3rd", "4th", "5th", etc.).

Expected Output:

Finishing position: 1

1st!

Finishing position: 2

2nd!

Finishing position: 4

4th!

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Solution

```
# insert a number
n:int = int(input("Insert finishing Position: "))

# check inserted number and return the position in cardinal form

# match statement
match n:

    # n=1
    case 1:
        print(f"{n}st!")

    # n=2
    case 2:
        print(f"{n}nd!")

    # n=3
    case 3:
        print(f"{n}rd!")

    # default case
    case _:
        print(f"{n}th!")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and OR/1

```
#http status (if/elif/else version)

http_status=200

if http_status==200 or http_status==201:
    print("Success")

elif http_status==400:
    print("Bad request")

elif http_status==404:
    print("Not Found")

elif http_status==500 or http_status==501:
    print("Server Error")
else:
    print("Unknown")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and OR/2

```
#http status (match statement version)

http_status=200

match http_status:

    case 200|201:
        print("Success")

    case 400:
        print("Bad Request")

    case 404:
        print("Not Found")

    case 500|501:
        print("Server Error")

    case _:
        print("Unknown")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and conditions

```
# insert a number
n:int = int(input("Insert a number: "))

match n:

    # case n is a positive and even number
    case n if n > 0 and n%2==0:
        print(f"{n} is positive and even")

    # case n is a positive and odd number
    case n if n > 0 and n%2!=0:
        print(f"{n} is positive and odd")

    # case n is a negative and even number
    case n if n < 0 and n%2==0:
        print(f"{n} is negative and even")

    # case n is negative and odd number
    case n if n < 0 and n%2!=0:
        print(f"{n} is negative and odd")

    # default case
    case _:
        print("The inserted number is 0!")
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and variables

```
# Define variables

g: str = "f"
age: int = 5

# Match statement

match (g, age):

    case ("f", 5):
        print("Piccola!")

    case ("m", 5):
        print("Piccolo!")

    case ("f", 10):
        print("Grande!")

    case ("m", 10):
        print("Gigante!")

    case _:
        print("Kinder Sorpresa!")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and lists

```
# Define a list

ingredients = ["pomodoro", "mozzarella", "basilico"]

# match statement

    match ingredients:

        case ["pomodoro", "mozzarella", "basilico"]:
            print("Puoi fare una Caprese!")

        case ["pasta", "pomodoro", *_]:
            print("Puoi fare la Pasta al Pomodoro!")

        case ["pane", "prosciutto", "formaggio"]:
            print("Puoi fare un Panino!")

        case _:
            print("Non saprei cosa consigliare…sperimenta!")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and dictionaries/1

```
# Define a dictionary

user =  {"nome": "Luca", "ruolo": "admin"}

# match statement

match user:

        case {"nome": name, "ruolo": "admin"}:
            print(f"Benvenuto amministratore {name}")

        case {"nome": name, "ruolo": "utente"}:
            print(f"Benvenuto utente {name}")

        case _:
            print("Ruolo non riconosciuto")
```

ITS ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and dictionaries/2

```
# Define a dictionary

user =  {"nome": "Luca", "ruolo": "admin"}

# match statement

match user:

        case {"nome": user["nome"], "ruolo": "admin"}:
            print(f"Benvenuto amministratore {user["nome"]}")

        case {"nome": user["nome"], "ruolo": "utente"}:
            print(f"Benvenuto utente {user["nome"]}")

        case _:
            print("Ruolo non riconosciuto")
```

ERROR!!!



ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and dictionaries/3

```
# Define a dictionary

user =  {"nome": "Luca", "ruolo": "admin"}

# match statement

match user:

        case {"nome": "Luca", "ruolo": "admin"}:
            print(f"Benvenuto amministratore {user["nome"]}")

        case {"nome": "Anna", "ruolo": "utente"}:
            print(f"Benvenuto utente {user["nome"]}")

        case _:
            print("Ruolo non riconosciuto")
```



ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and dictionaries/4

```
# Define a dictionary

user =  {"nome": "Luca", "ruolo": "admin"}

# match statement

match user:

        case {"nome": name, "ruolo": "admin"}:
            print(f"Benvenuto amministratore {user["nome"]}")

        case {"nome": name, "ruolo": "utente"}:
            print(f"Benvenuto utente {name}")

        case _:
            print("Ruolo non riconosciuto")
```



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and tuples/1

```
# Define a tuple

point = (3,5)

# match statement

match point:

    case (0, 0):
        print("Origine")

    case (x, 0):
        print(f"Punto sull'asse X: ({x}, 0)")

    case (0, y):
        print(f"Punto sull'asse Y: (0, {y})")

    case _:
        print(f"Punto generico: {point}")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and tuples/2

```
# Define a tuple

point = (3,5)

# match statement

match point:

    case (0, 0):
        print("Origine")

    case (point[0], 0):
        print(f"Punto sull'asse X: ({point[0]}, 0)")

    case (0, point[1]):
        print(f"Punto sull'asse Y: (0, {point[1]})")

    case _:
        print(f"Punto generico: {point}")
```

ERROR!!!



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and tuples/3

```
# Define a tuple

point = (3,5)

# match statement

match point:

    case (0, 0):
        print("Origine")

    case (1, 0):
        print(f"Punto sull'asse X: ({point[0]}, 0)")

    case (0, 2):
        print(f"Punto sull'asse Y: (0, {point[1]})")

    case _:
        print(f"Punto generico: {point}")
```



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Match Statement and tuples/4

```
# Define a tuple

point = (3,5)

# match statement

match point:

    case (0, 0):
        print("Origine")

    case (x, 0):
        print(f"Punto sull'asse X: ({x}, 0)")

    case (0, y):
        print(f"Punto sull'asse Y: (0, {y})")

    case _:
        print(f"Punto generico: {(point[0],point[1]) }")
```



ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY